第4回: JavaScript基礎 — TODOアプリに動きをつける

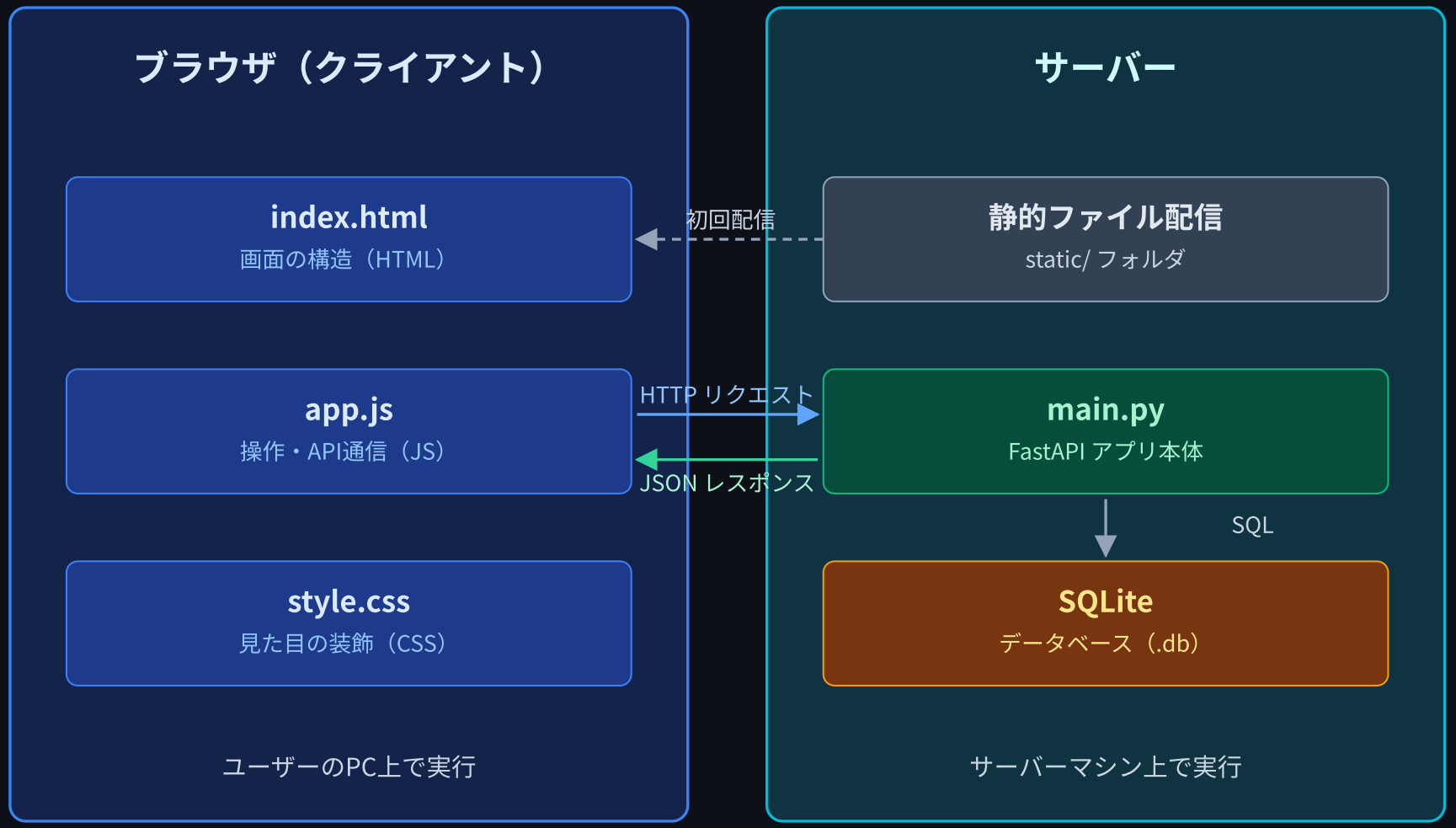
Webアプリケーション基礎 2026

今日のゴール

JavaScriptを使って、TODOアプリに **動き** をつける

- TODOの **追加**
- TODOの **完了切り替え**
- TODOの **削除**

> フロントエンド（ブラウザ）だけで動作する版を完成させます


ブラウザ（クライアント）
index.html
画面の構造（HTML）
app.js
操作・API通信（JS）
style.css
見た目の装飾（CSS）
ユーザーのPC上で実行
初回配信
HTTP リクエスト
JSON レスポンス
サーバー
静的ファイル配信
static/ フォルダ
main.py
FastAPI アプリ本体
SQL
SQLite
データベース（.db）
サーバーマシン上で実行

今日の流れ

前半

- JavaScriptとは
- 変数とデータ型
- 関数・条件分岐・ループ

後半

- DOM操作
- TODOアプリのJS実装（1）追加機能
- TODOアプリのJS実装（2）完了・削除

前回までの振り返り

> 今日ここ！ → TODOアプリに「動き」をつける

現状の問題: ボタンを押しても何も起きない...

HTML・CSS・JavaScriptの役割

技術	役割
HTML	構造・骨格
CSS	見た目・装飾
JavaScript	**動き・操作**

> JavaScriptがあって初めて、ユーザーの操作に「反応」できるようになります

セクション1: JavaScriptとは

JavaScriptとは？

- **ブラウザ上で動くプログラミング言語**
- 1995年に誕生、現在はWeb開発の必須技術
- HTMLページに「動き」や「インタラクション」を追加する

実行場所

> 今回は全てブラウザ上で動作します

JavaScriptの書き方（3つの方法）

1. HTMLファイル内に直接書く（インライン）

```
<button onclick="alert('クリックされました！')">押してね</button>
```

2. scriptタグ内に書く

```
<script>
  console.log("Hello, JavaScript!");
</script>
```

3. 外部ファイルから読み込む（推奨）

```
<script src="app.js"></script>
```

console.log — 最初の一歩

`console.log()` はブラウザの **開発者ツール（Console）** にメッセージを表示する命令です。

```
console.log("Hello, JavaScript!");
console.log(2 + 3);           // 5
console.log("TODO" + "アプリ"); // TODOアプリ
```

開発者ツールの開き方

- **Windows/Linux:** `F12` または `Ctrl + Shift + I`
- **Mac:** `Cmd + Option + I`
- 「Console」タブを選択

scriptタグの配置場所

```html
<!DOCTYPE html>
<html>
<head>
  <title>サンプル</title>
</head>
<body>
  <h1>Hello</h1>
  <p>本文</p>

  <!-- bodyの閉じタグの直前に配置するのが基本 -->
  <script src="app.js"></script>
</body>
</html>
```

> **理由:** HTMLが全て読み込まれた後にJSが実行されるため、要素を確実に操作できる

実習1: Console で遊んでみよう

1. ブラウザで任意のページを開く
2. 開発者ツールを開き、**Console** タブを選択
3. 以下を順番に入力して Enter で実行:

```
console.log("Hello!")
2 + 3
10 * 5
"Web" + "アプリ"
"TODOの数は" + 3 + "個です"
```

4. 計算結果や文字列がどう表示されるか確認しましょう

セクション2: 変数とデータ型

変数とは？

データに **名前** をつけて保存しておく箱のようなもの。

let — 後から値を変更できる変数

```
let count = 0;
count = 1;          // OK: 値を変更できる
count = count + 1; // OK: 2になる
```

const — 値を変更できない変数（定数）

```
const appName = "TODO App";
appName = "New App"; // エラー！ constは再代入できない
```

> **基本方針:** まず `const` を使い、変更が必要なら `let` に変える

データ型

JavaScriptの主なデータ型:

```
// 文字列（String）— クォーテーションで囲む
const name = "太郎";
const greeting = 'こんにちは';

// 数値（Number）— 整数も小数も同じ
const age = 20;
const pi = 3.14;

// 真偽値（Boolean）— true か false
const isDone = false;
const isActive = true;
```

配列（Array）

複数の値をまとめて管理するデータ構造。

```
// 配列の作成
const fruits = ["りんご", "バナナ", "みかん"];

// 要素へのアクセス（0から始まる）
console.log(fruits[0]); // "りんご"
console.log(fruits[1]); // "バナナ"

// 要素数
console.log(fruits.length); // 3

// 末尾に追加
fruits.push("ぶどう");
console.log(fruits.length); // 4
```

オブジェクト（Object）

名前付きのデータをまとめて管理する。

```
// オブジェクトの作成
const todo = {
  title: "買い物に行く",
  done: false
};

// プロパティへのアクセス
console.log(todo.title); // "買い物に行く"
console.log(todo.done);  // false

// 値の変更
todo.done = true;
console.log(todo.done);  // true
```

> 次回以降学ぶ **JSON** と非常によく似た形式です

TODOを配列で表現する

```
const todos = [
  { title: "買い物に行く", done: false },
  { title: "レポートを書く", done: true },
  { title: "本を読む", done: false }
];

console.log(todos[0].title); // "買い物に行く"
console.log(todos[1].done);  // true
console.log(todos.length);   // 3

// 新しいTODOを追加
todos.push({ title: "部屋を掃除する", done: false });
console.log(todos.length);   // 4
```

実習2: 変数と配列を操作しよう

Console で以下を入力して実行:

```
// 1. 変数を宣言
let count = 0;
console.log(count);
count = count + 1;
console.log(count);

// 2. 文字列の定数を宣言
const myName = "（自分の名前）";
console.log(myName + "のTODOリスト");

// 3. TODOの配列を作成
const todos = ["買い物", "勉強", "運動"];
console.log(todos);
console.log(todos.length);
```

実習2: 変数と配列を操作しよう（続き）

```
// 3. 配列にTODOを追加
todos.push("読書");
console.log(todos);

// 4. オブジェクトの配列でTODOを表現
const todoList = [
  { title: "買い物", done: false },
  { title: "勉強", done: true }
];
console.log(todoList[0].title);
```

セクション3: 関数・条件分岐・ループ

関数とは？

処理をまとめて名前をつけたもの。何度でも呼び出せる。

function宣言

```
function greet(name) {
  console.log("こんにちは、" + name + "さん！");
}

greet("太郎"); // こんにちは、太郎さん！
greet("花子"); // こんにちは、花子さん！
```

アロー関数（ES6以降の書き方）

```
const greet = (name) => {
  console.log("こんにちは、" + name + "さん！");
};

greet("太郎"); // こんにちは、太郎さん！
```

関数の戻り値

```
// 戻り値のある関数
function add(a, b) {
  return a + b;
}

const result = add(3, 5);
console.log(result); // 8

// アロー関数版
const multiply = (a, b) => {
  return a * b;
};

console.log(multiply(4, 6)); // 24
```

条件分岐（if / else）

```
const score = 75;

if (score >= 80) {
  console.log("優秀です！");
} else if (score >= 60) {
  console.log("合格です");
} else {
  console.log("もう少し頑張りましょう");
}
// → "合格です"
```

TODOでの活用例

```
const todo = { title: "買い物", done: true };

if (todo.done) {
  console.log(todo.title + " は完了済みです");
} else {
  console.log(todo.title + " は未完了です");
}
```

ループ（for / forEach）

for文

```
const todos = ["買い物", "勉強", "運動"];

for (let i = 0; i < todos.length; i++) {
  console.log(i + ": " + todos[i]);
}
// 0: 買い物
// 1: 勉強
// 2: 運動
```

forEach（配列のメソッド）— こちらを多用します

```
todos.forEach((todo) => {
  console.log(todo);
});
```

実習3: 関数とループを使おう

Console で実行した後、app.jsファイルを作成します

```
// 1. TODOリストを表示する関数
const todos = [
  { title: "買い物に行く", done: false },
  { title: "レポートを書く", done: true },
  { title: "本を読む", done: false }
];

function showTodos(todoList) {
  todoList.forEach((todo) => {
    const status = todo.done ? "[完了]" : "[未完了]";
    console.log(status + " " + todo.title);
  });
}

showTodos(todos);
```

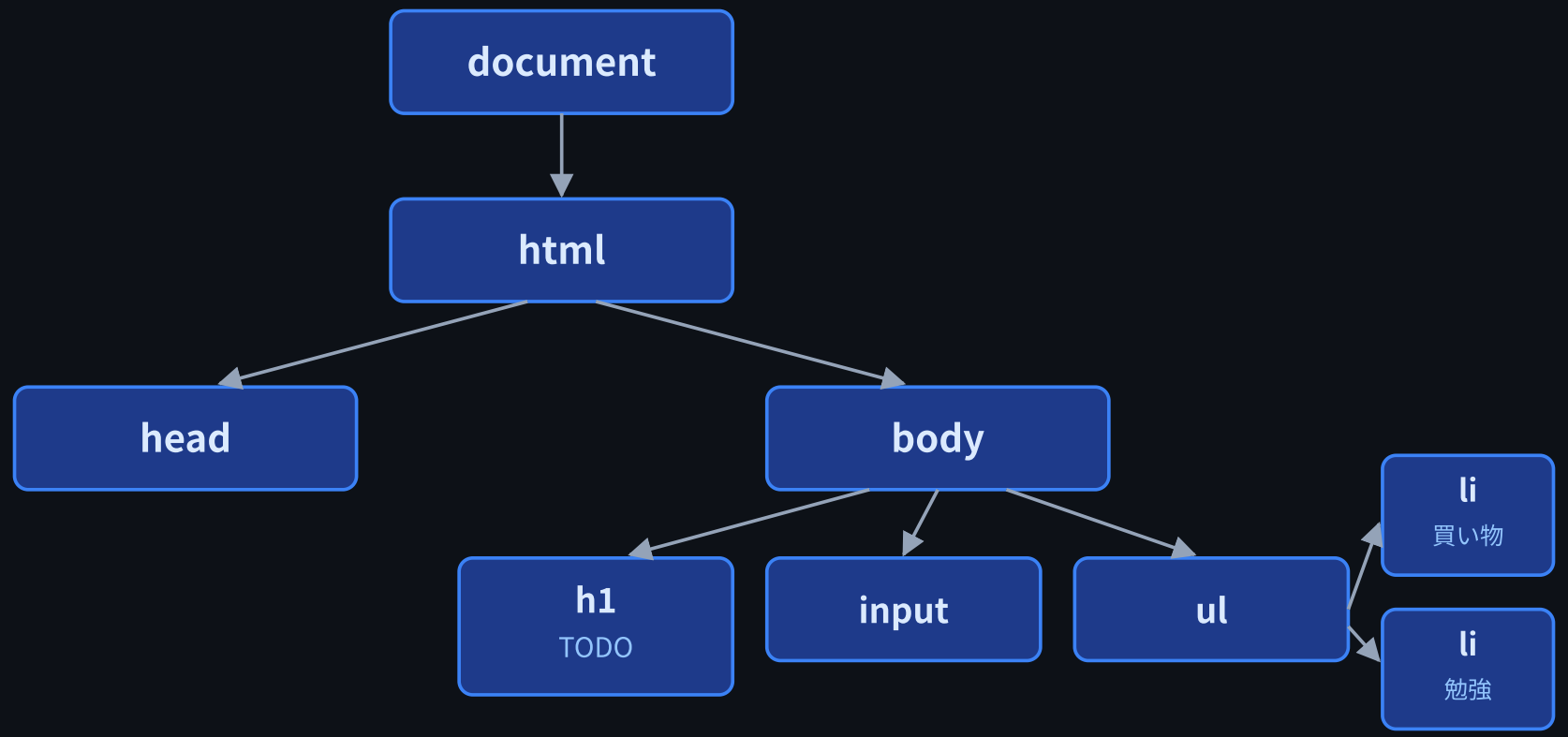
2. エディタで `app.js` を作成し、上記コードを貼り付け
3. `todo.html` に `<script src="app.js"></script>` を追加
4. ブラウザで開いてConsoleを確認

セクション4: DOM操作

DOMとは？

DOM（Document Object Model） = HTMLをJavaScriptから操作するための仕組み

ブラウザはHTMLを読み込むと、内部的に「ツリー構造」のデータに変換します。



> この **ツリー構造** を JavaScript から操作するのが **DOM操作** です

要素の取得

```
// IDで取得（最もよく使う）
const title = document.getElementById("app-title");

// CSSセレクタで取得（1つ）
const button = document.querySelector(".add-btn");

// CSSセレクタで取得（すべて）
const items = document.querySelectorAll(".todo-item");
```

対応するHTML

```
<h1 id="app-title">TODO App</h1>
<button class="add-btn">追加</button>
<li class="todo-item">買い物</li>
<li class="todo-item">勉強</li>
```

要素の作成と追加

```
// 新しい要素を作成
const newItem = document.createElement("li");

// テキストを設定
newItem.textContent = "新しいTODO";

// 既存の要素の子要素として追加
const list = document.getElementById("todo-list");
list.appendChild(newItem);
```

操作前後のイメージ

```
操作前:  <ul id="todo-list">
            <li>買い物</li>
         </ul>

操作後:  <ul id="todo-list">
            <li>買い物</li>
            <li>新しいTODO</li>   ← 追加された！
         </ul>
```

テキストの設定方法

textContent（安全）

```
const element = document.getElementById("message");
element.textContent = "こんにちは！";
```

innerHTML（注意が必要）

```
const element = document.getElementById("message");
element.innerHTML = "<strong>太字</strong>のテキスト";
```

> **注意:** `innerHTML` はHTMLタグを解釈するため、ユーザー入力をそのまま入れると **セキュリティリスク（XSS）** があります。基本は `textContent` を使いましょう（第8回で詳しく学びます）。

イベントリスナー

ユーザーの操作（クリック、入力など）を検知して処理を実行する仕組み。

```
const button = document.getElementById("add-btn");

button.addEventListener("click", () => {
  console.log("ボタンがクリックされました！");
});
```

よく使うイベント

イベント名	発生タイミング
`click`	クリックされた時
`submit`	フォームが送信された時
`input`	入力欄の値が変わった時
`keydown`	キーが押された時

実習4: DOM操作をやってみよう

dom-example.html を使って操作してみましょう

1. `session04/exercise/dom-example.html` を開き、`<script>` タグの中に以下のコードを追加してください

```
// 要素の取得 アイテムを追加するボタンとリストを取得
const addItemBtn = document.getElementById("add-item-btn");
const itemList = document.getElementById("item-list");

// 「追加」ボタンをクリックしたときの処理
addItemBtn.addEventListener("click", () => {
  // 新しいli要素を作成して追加
  const newItem = document.createElement("li");
  newItem.textContent = "新しいアイテム";
  itemList.appendChild(newItem);
});
```

2. プレビューで動作を確認
3. `newItem.textContent` の中身を変えてみて、追加されるテキストを変えてみましょう

セクション5: TODOアプリのJS実装（1）追加機能

TODOアプリの処理の流れ

入力値の取得

```html
<form id="todo-form" class="input-area">
  <input
    type="text"
    id="todo-input"
    class="todo-input"
    placeholder="新しいTODOを入力..."
    autocomplete="off"
  >
  <button type="submit" class="todo-button">追加</button>
</form>
```

```javascript
const todoForm = document.getElementById("todo-form");
const todoInput = document.getElementById("todo-input");
todoForm.addEventListener("submit", (event) => {
  event.preventDefault();          // ページリロードを防止
  const text = todoInput.value;    // 入力欄の値を取得
  console.log("入力された値:", text);
  todoInput.value = "";            // 入力欄をクリア
});
```

> `input.value` で入力欄のテキストを取得できます

event.preventDefault()

フォームの `submit` イベントを使う場合、デフォルトではページがリロードされてしまいます。

```
const todoForm = document.getElementById("todo-form");
const todoInput = document.getElementById("todo-input");

todoForm.addEventListener("submit", (event) => {
  event.preventDefault(); // デフォルト動作（リロード）を防止！

  const text = todoInput.value.trim();

  if (text === "") return; // 空文字なら何もしない

  // TODO追加処理...
  todoInput.value = "";
  todoInput.focus();
});
```

> `event.preventDefault()` は非常に重要です。フォーム送信時のリロードを防ぎます。

DOM再描画の仕組み

配列を更新したら、リスト全体を再描画（描き直し）します。

```
let todos = [];
const todoList = document.getElementById("todo-list");

function render() {
  todoList.innerHTML = ""; // リストを一旦空にする

  todos.forEach((todo) => {
    const li = document.createElement("li");
    li.className = "todo-item";
    li.textContent = todo.title;
    todoList.appendChild(li);
  });
}
```

> `render()` を呼ぶと、todos配列の内容でリスト全体を描き直します

TODO追加の全体像

```
let todos = [];
const todoList = document.getElementById("todo-list");

function addTodo(title) {
  if (title === "") return; // 空文字なら何もしない
  todos.push({
    title: title,
    done: false
  });
  render(); // 画面を再描画
}

function render() {
  todoList.innerHTML = "";

  todos.forEach((todo) => {
    const li = document.createElement("li");
    li.className = "todo-item";
    li.textContent = todo.title;
    todoList.appendChild(li);
  });
}
```

実習5: TODO追加機能を実装しよう

exercise/app.js を編集します

1. `exercise/todo.html` をブラウザで開く
2. `exercise/app.js` の `addTodo` 関数と `render` 関数を実装
3. 動作確認:
   - 入力欄にテキストを入力
   - 「追加」ボタンをクリック
   - リストにTODOが追加される
   - 入力欄がクリアされる
4. 複数のTODOを追加してみよう

セクション6: TODOアプリのJS実装（2）完了・削除

完了切り替え — チェックボックスで切り替え

チェックボックスの状態変更で、完了状態を表現します。

```
// CSSで完了状態のスタイルを定義
// .todo-item.done .todo-title { text-decoration: line-through; opacity: 0.6; }

function toggleTodo(index) {
  todos[index].done = !todos[index].done; // true ⇔ false を切り替え
  render();                               // 画面を再描画
}

// render内でチェックボックスを作成
const checkbox = document.createElement("input");
checkbox.type = "checkbox";
checkbox.className = "todo-checkbox";
checkbox.checked = todo.done;
checkbox.addEventListener("change", () => toggleTodo(index));
```

表示イメージ

```
チェック前:  □ 買い物に行く
チェック後:  ☑ 買い物に行く   ← 取り消し線＋薄くなる
```

削除機能

各TODOに削除ボタンをつけ、クリックで配列から削除して再描画します。

```
function deleteTodo(index) {
  todos.splice(index, 1); // 配列からindex番目を削除
  render();               // 再描画
}

// render内で削除ボタンを作成
const deleteBtn = document.createElement("button");
deleteBtn.className = "delete-button";
deleteBtn.textContent = "削除";
deleteBtn.addEventListener("click", () => deleteTodo(index));

li.appendChild(deleteBtn);
```

完了トグルと削除を組み合わせたrender関数

```
function render() {
  todoList.innerHTML = "";

  todos.forEach((todo, index) => {
    const li = document.createElement("li");
    li.className = "todo-item" + (todo.done ? " done" : "");

    const label = document.createElement("label");
    label.className = "todo-label";

    const checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.className = "todo-checkbox";
    checkbox.checked = todo.done;
    checkbox.addEventListener("change", () => toggleTodo(index));
```

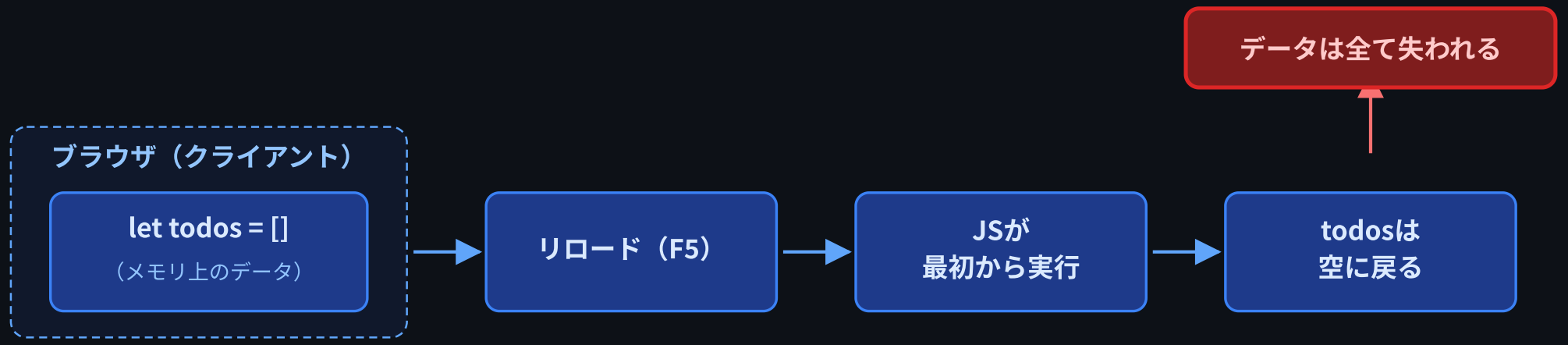
完了トグルと削除を組み合わせたrender関数（続き）

```
    const span = document.createElement("span");
    span.className = "todo-title";
    span.textContent = todo.title;

    const deleteBtn = document.createElement("button");
    deleteBtn.className = "delete-button";
    deleteBtn.textContent = "削除";
    deleteBtn.addEventListener("click", () => deleteTodo(index));

    label.appendChild(checkbox);
    label.appendChild(span);
    li.appendChild(label);
    li.appendChild(deleteBtn);
    todoList.appendChild(li);
  });
}
```

重大な問題: リロードするとデータが消える！

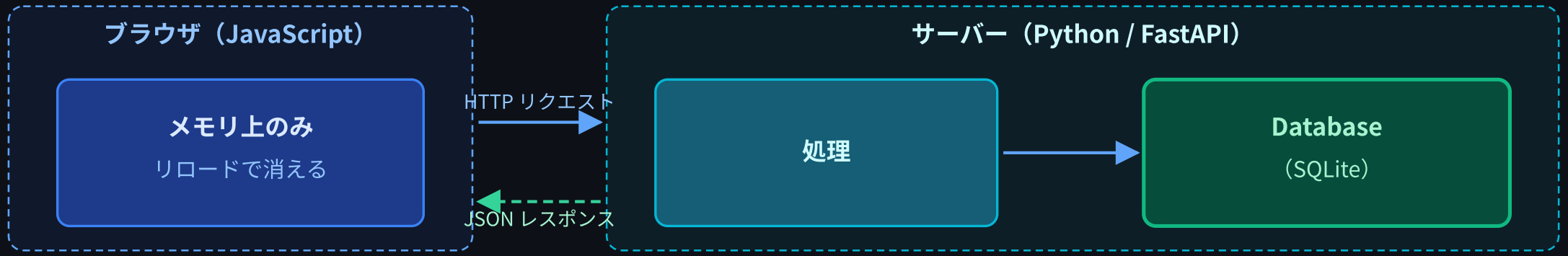
```
1. TODOを5個追加した
2. ページをリロード（F5）
3. ... 全部消えた！！！
```

なぜ？



> JavaScriptの変数はブラウザの **メモリ上** にしか存在しない。
> リロードやブラウザを閉じると、メモリはクリアされる。

どうすればデータを保存できる？



> データが永続化される！

> **次回予告:** Python & FastAPIでサーバーを作り、データを保存できるようにします！

実習6: 完了・削除機能を実装しよう

1. `exercise/app.js` の `render` 関数に完了トグルと削除機能を追加
2. 動作確認:
   - TODOをクリック → 取り消し線が表示/非表示
   - 「削除」ボタンをクリック → TODOが削除される
3. **リロードテスト:** TODOを追加後、F5でデータが消えることを確認
4. Git commit & push

```
git add .
git commit -m "第4回: JavaScript基礎 - TODOアプリのフロントエンド実装"
git push
```

今日のまとめ

学んだこと	内容
JavaScript基礎	console.log、変数、データ型
配列・オブジェクト	TODOデータの管理方法
関数・条件分岐・ループ	処理のまとめ方、繰り返し
DOM操作	HTML要素の取得・作成・追加
イベントリスナー	クリック操作への応答
TODOアプリ実装	追加・完了・削除機能

第4回終了時点のTODOアプリ

```
フロントエンド完成！ だが...

  リロードするとデータが全部消える

→ 「サーバーにデータを保存する仕組みが必要」
→ 次回: Python & FastAPI入門でサーバーサイドを学びます！
```

次回予告: 第5回 Python & FastAPI入門

- **なぜバックエンドが必要か** を再確認
- **Python** の基礎（変数、関数、ループ）
- **FastAPI** でWebサーバーを起動
- **JSON** でデータをやり取り
- TODOリストをサーバーから返す API を作成

> 実行場所がブラウザからサーバーに変わります！

提出物

実習で穴埋め・実装を完了させたファイルをフォームから提出してください:

1. `app.js` のGitHubのURL
    - 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session04/exercise/app.js`
2. `dom-example.html` のGitHubのURL
    - 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session04/exercise/dom-example.html`